Specifying Embedded Systems with Statecharts and Z: An Agenda for Cyclic Software Components

Wolfgang Grieskamp* Maritta Heisel† Heiko Dörr‡

October 14, 1998

Abstract

The application of formal techniques can contribute much to the quality of software, which is of utmost importance for safety-critical embedded systems. These techniques, however, are not easy to apply. In particular, methodological guidance is often unsatisfactory. We address this problem by the concept of an *agenda*. An agenda is a list of activities to be performed for solving a task in software engineering. Agendas used to support the application of formal specification techniques provide detailed guidance for specifiers, templates of the used specification language that only need to be instantiated, and application independent validation criteria. We apply the agenda approach to a particular class of embedded safety-critical systems, the formal specification of which has been investigated in the case-studies of the German Espress project during the last two years.

1 Introduction

Every software-based system potentially benefits from the application of formal techniques. For the development of mission or even safety-critical embedded systems, however, their use is of particular advantage, because the potential damage operators and developers have to envisage in case of malfunction may be much worse than the additional costs of applying formal techniques in system development.

A major drawback of formal techniques is that they are not easy to apply for the average software engineer. Besides the facts that users of formal techniques need an appropriate education and have to deal with lots of details, they are often left alone with a mere formalism without any guidance on how to use it. Hence, *methodological support* is a key issue to bring formal techniques into practice.

Methodological support for specification development must be abstract enough to cover a significant range of applications, but also detailed enough to provide real guidance to developers. To achieve this aim, the problem-related fine-grained knowledge acquired by experienced developers needs to be made explicit and represented in a way that supports its reuse. *Agendas*, as introduced in [20, 21], provide a concept for representing methodological process knowledge for particular software architectures on a relatively fine-grained level of detail.

*Technische Universität Berlin, Institut für Kommunikations- und Softwaretechnik, Sekr. FR5-13, Franklinstr. 28/29, D-10587 Berlin, Germany, email: wg@cs.tu-berlin.de

†Otto-von-Guericke-Universität Magdeburg, Fakultät für Informatik, Institut für Verteilte Systeme, D-39016 Magdeburg, Germany, email: heisel@cs.uni-magdeburg.de

‡Daimler-Benz AG, Forschung Systemtechnik, Alt-Moabit 96a, D-10569 Berlin, Germany, email: Heiko.Doerr@DBAG.Bln.DaimlerBenz.com

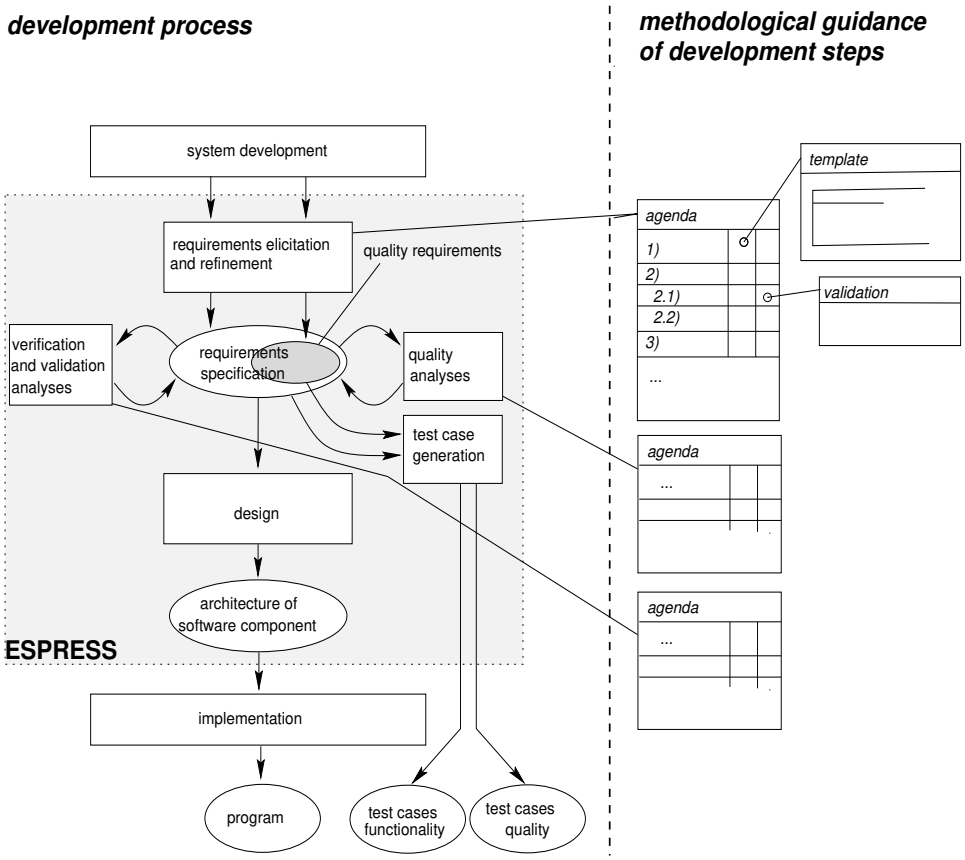


Figure 1: Basic ESPRESS process model

In this paper, we demonstrate the application of the concept of agendas to a particular class of embedded safety-critical systems. The architecture we study is that of a *cyclic software component* – a piece of software in a technical system which is triggered in regular time intervals by its environment to compute output values (usually commands to some actuators) from given input values (usually sensor values), and an internal state. The agenda we present for this architecture is condensed from case studies performed in the ESPRESS project[1].

Figure 1 shows the basic ESPRESS process model. The agenda presented in this paper guides the development of a requirements specification. Such a requirements specification is further validated and serves as a basis for safety analyses, test case generation, and software design.

Our agenda for cyclic software components elaborates on two particular aspects of embedded systems, motivated by the demands of the ESPRESS application context. First, special care is taken to accurately develop the embedding of the software in its surrounding technical system. Second, *quality requirements* such as high-level or safety-related conditions which have to be guaranteed by the software need to be treated systematically. The general ESPRESS methodology requires that – if possible – these requirements are specified as *properties*, which have to be logical *consequences* of an explicitly constructed model of the software. The redundancy introduced by this approach increases the potential for checking the consistency

[1]The ESPRESS project is a cooperation of industry and research institutes funded by the German ministry BMBF ("Förderschwerpunkt Softwaretechnologie") from 1995 to 1998.

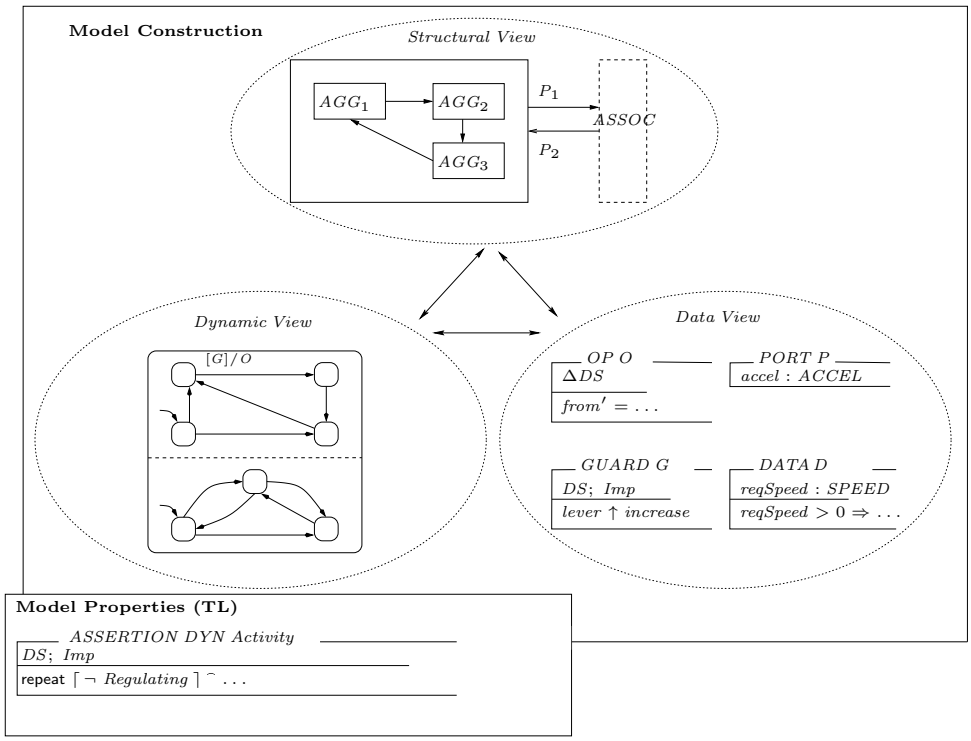


Figure 2: The Notations of $\mu\mathcal{SZ}$

of the formal specification. It also supports a development situation where model construction and model properties are defined by different teams.

Outline of this Paper. Section 2 introduces the Espress notations and tools we use. In Section 3 we discuss the concept of an agenda in general. Section 4 presents the concrete agenda for cyclic software components and its application to the case study of an intelligent cruise control system. In Section 5, we discuss related work, before concluding in Section 6.

2 Notations and Tools

We use the Espress notation $\mu\mathcal{SZ}$ [2, 4] to express the specifications developed with our agenda for cyclic software components. This notation provides a semantically well-defined combination of the Statemate languages [16] (namely statecharts and activity charts), the formal specification language Z [41], and an extension of Z by temporal interval logic [5]. The Statemate languages and Z have been chosen for Espress because of their relevance in industrial contexts and their fairly good tool support. Interval logic is used because of its relation to nondeterministic automata and regular expressions, concepts which are familiar to engineers.

2.1 Outline of $\mu\mathcal{SZ}$

A $\mu\mathcal{SZ}$ specification consists of so called *process class* definitions, which cluster pieces of the constituting notations. A process class describes the interfaces and behavior of a set of processes, the instances of the class. Figure 2 shows the components of a process class:

- A specification of the process *structure*, i.e. aggregation and association with instances of other classes, and the data-flow between these instances. This is represented by Statemate's activity charts [16].
- A specification of the *data space* of the process, including constant definitions, data shared with the environment, internal data, and transformations of data. The data space is specified using conventional Z – except that schemas may have associated a certain *role* in the context of the process class. Schema roles include

  PORT – declares data shared by the process with the environment

  DATA – declares data private to the process

  INIT – constraints the initial value of the data space

  GUARD – specifies a condition on the data space

  OP – specifies an operation (a transformation of the data state)

  PROPERTY – specifies an invariant of the data space
- A specification of the *dynamic behavior* of the process, based on statecharts. The transitions of statecharts are labelled with pairs of *GUARD* and *OP* schemas. A transition is taken if its source state is active, and the guard schema is true on the current data state. Its effect is a transformation of the data state as specified by the *OP* schema.
- A specification of selected *dynamic* properties which are asserted for the process, expressed by an embedding of discrete temporal interval logic into Z. These are given in special schemas tagged with the phrase *ASSERTION DYN*.

2.2 Semantics of $\mu\mathcal{SZ}$

The semantic foundation of $\mu\mathcal{SZ}$ is not subject of this paper. Our agenda for cyclic software components expressed in $\mu\mathcal{SZ}$ can be understood without a deeper understanding of the semantics of $\mu\mathcal{SZ}$. For those readers interested in this subject, we informally describe some aspects of the semantics below. Basic knowledge of Z and Statecharts is assumed.

Data Space. Let *Data* denote the conjunction of all the schemas with role *DATA* in a process class, *Port* with role *PORT*, *Property* with role *PROPERTY*, and *Init* with role *INIT*. The *data space* of a process class is given by the schema *DataSpace*, the bindings of which represent the admissible data state at any point in time:

$$DataSpace \mathrel{\widehat{=}} [Port;\ Data \mid Property]$$

Let $\text{seq}_\infty\ \alpha$ represent a type constructor for the set of infinite sequences of elements of α. Then the admissible *data traces* of a process class, as induced by the data space specification, are defined as:

$$DataTraces == \{t : \text{seq}_\infty\ DataSpace \mid t\,1 \in Init\}$$

Statecharts Step Algorithm. The traces of the data view are a superset of the traces which result from an adaption of the statechart step algorithm to our model. We give an informal explanation of this adaption, which is the major gluing point of our semantic integration of Statecharts and Z. For a detailed description of the conventional statechart step algorithm see [18].

The input for the step algorithm is the current process status σ, i.e. the set of Statechart-states the process currently resides in and a binding of the schema *DataSpace*. Using this information, the set of *enabled transitions* is computed. A transition is enabled if all of its source states are active and the guard of its label is true for the data binding. From this set, a *maximal non-conflicting*[2] set is computed by removing those transitions that are in conflict with an enabled transition of higher priority[3], and non-deterministically removing transitions that have equal priority with conflicting transitions.

From the maximal set of non-conflicting transitions, the transformation to perform on the data state is synthesised by merging the Z operations of the non-conflicting transitions. The main complication here is to treat *racing*, the situation where two or more operations of the non-conflicting transitions write the same variable. To this end, the operations are grouped according to common variables they write: whenever two operations have a common variable x' in their signature, they are joined to the same group. Let Op_{i_j} be the operations such that j ranges over the groups and i_j over the operations in each group. Then the merged data state transformation is defined in Z as follows:

$$Op_{fire} \mathrel{\widehat=} \bigwedge_{j=1}^{n} \bigvee_{i=1}^{n_j} Op_{i_j}$$

Thus inside each group one operation is chosen nondeterministically, whereas the groups altogether are executed in parallel. The nondeterministic choice inside a group reflects racing resolution on the level of Z operations.

To describe the actual data transformation associated with a step, two further details need to be considered: ensuring that invariants specified in the data view of a process class are treated, and ensuring that those variables which are not written in a step keep persistent. Let W denote the set of variables not written by Op_{fire} (i.e. those variables whose primed version do not appear in the signature) and which are not declared to be derived[4]:

$$Op_{step} \mathrel{\widehat=} Op_{fire} \wedge \Delta DataSpace \wedge \Xi(DataSpace \setminus W)$$

It is required that the precondition of the synthesised operation Op_{step} be true under the current variable binding. Otherwise the step algorithm is undefined for the given σ. Thus the step algorithm is a partial, nondeterministic mapping (relation) from process states to process states. Since the transitions are non-conflicting, and $\mathsf{pre}\ Op_{step}$ needs to hold for σ, it is ensured that the new process status σ' – if existent – is valid according to the statecharts structure and is an element of *DataSpace*.

Temporal Model Properties. For the temporal model properties, a trace semantics is used. The correctness of a model according to its temporal properties is stated by $ModelTraces \Rightarrow TemporalTraces$ (or $ModelTraces \subseteq TemporalTraces$, resp.), where *ModelTraces* are those traces which can be constructed from applying the step algorithm to the initial states of a process class. A more detailed account of the temporal semantics we use is given in [5].

[2] Two transitions are said to be in conflict if the intersection of the set of states they are exiting is non-empty.

[3] A transition has a higher priority than another transition if its context is an ancestor of the other transition's context.

[4] $\mu\mathcal{SZ}$ allows variables of the data space to be flagged as *derived* from the value of other variables. The value of these variables is implicitly defined in each step from the invariants of the data view. Semantically this just means that those variables are not subject of implicit persistency.

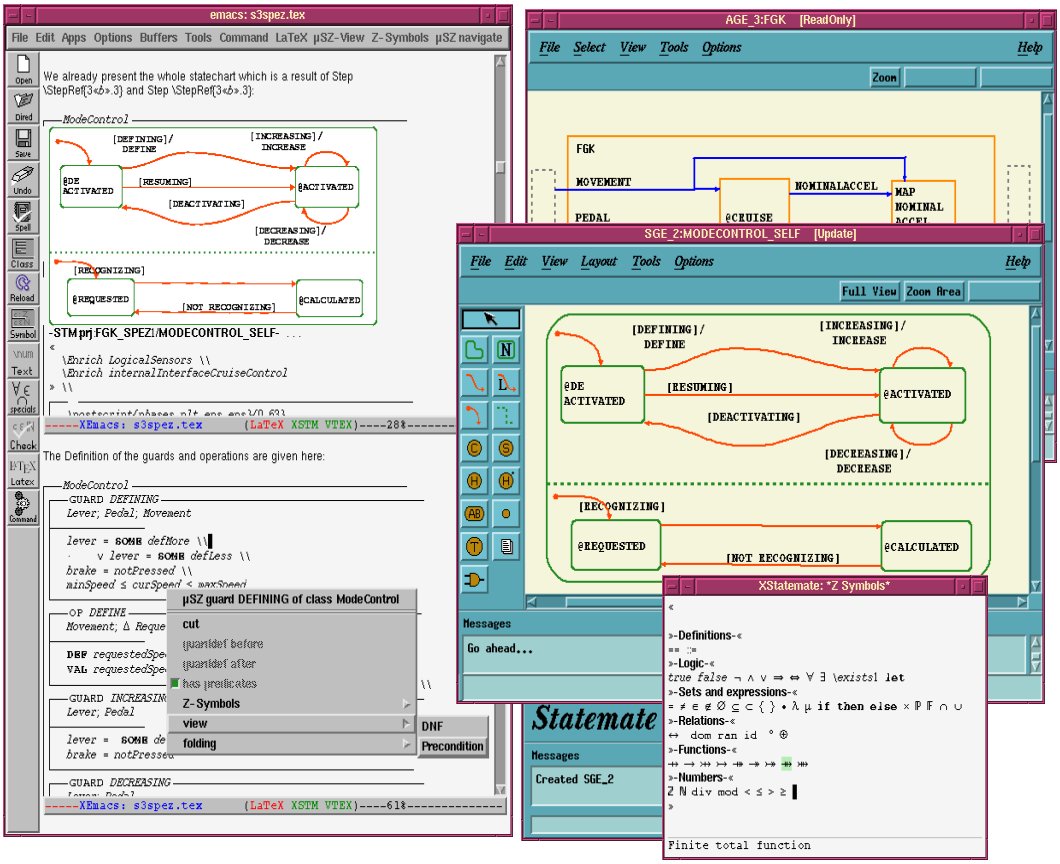


Figure 3: Screenshot of a Session with the ESPRESS Tool Environment

2.3 Tool support for $\mu\mathcal{SZ}$

As the integrated notations of $\mu\mathcal{SZ}$ are syntactically and semantically preserved, tools for them can be reused. "Glueware" has been designed in ESPRESS to constitute a tool environment which integrates existing tools such as Statemate, the deduction system Isabelle [34], the model checker SMV [30], the extendable text editor XEmacs, and newly written tools developed in ESPRESS, such as the embedding of Z in the higher-order logic of Isabelle, $\mathcal{HOL}$-$\mathcal{Z}$ [28], a type checker for $\mu\mathcal{SZ}$, a compiler for Z, and others. In the resulting environment, whose principal design is described in [6], $\mu\mathcal{SZ}$ specifications can be edited, browsed, analysed by deduction and model checking, and executed for the purpose of animation. For the agenda we present in this paper, the following capabilities are of particular interest:

- With $\mathcal{HOL}$-$\mathcal{Z}$ we can derive preconditions of operations, check for the existence of initial states and the consistency of invariants, and analyse the completeness of transitions of statecharts.
- With SMV we can model-check the validity of temporal model properties w.r.t. the constructed model.

The screenshot in Figure 3 shows a session with the ESPRESS tool environment, working on the specification of an intelligent cruise control system, which has been developed using the agenda for cyclic software components we present in this paper. The charts edited under Statemate are automatically mirrored in the view of the

$\mu\mathcal{SZ}$ specification edited under XEmacs. The analysis tools are running in the background, and can be activated by context-sensitive popup-menus associated with the $\mu\mathcal{SZ}$ text.

3 Agendas

Software development comprises a number of development activities, the result of each of which is an artifact, such as a requirements document, a formal specification, program code, and test cases. Experienced software engineers have over time acquired problem-related fine-grained knowledge how to perform the various development activities.

To date, such expert knowledge is rarely made explicit. This forces each software engineer to gain experience from scratch. Previously acquired knowledge is not reused to support software processes and not employed to educate novices.

An *agenda* is a means to explicitly represent software development knowledge. It gives guidance on how to perform a specific software development activity. Agendas can be used for structuring quite different activities in different contexts. We have set up and used agendas that support requirements engineering, specification acquisition, software design using architectural styles, and developing code from specifications [19]. Agendas are especially suitable to support the application of formal techniques in software engineering.

An agenda is a list of steps to be performed when carrying out some task in the context of software engineering. The result of the task will be a document expressed in a certain language. Agendas contain informal descriptions of the steps. With each step, *templates* of the language in which the result of the task is expressed are associated. The templates are instantiated when the step is performed. The steps listed in an agenda may depend on each other. Usually, they will have to be repeated to achieve the goal, similar to the general process proposed by the spiral model of software engineering. Agendas are presented as tables, see Fig. 5. Agendas may be nested, and we call the "super-steps" *stages* (see, e.g., Fig. 4).

Agendas are not only a means to guide software development activities. They also support quality assurance because the steps of an agenda may have validation conditions associated with them. These validation conditions state necessary conditions that the artifact must fulfill in order to serve its purpose properly. When formal techniques are applied, some of the validation conditions can be expressed and proven in a formal way. Since the validation conditions that can be stated in an agenda are necessarily application independent, the developed artifact should be further validated with respect to application dependent needs.

Working with agendas proceeds as follows: first, the software engineer selects an appropriate agenda for the task at hand. Usually, several agendas will be available for the same development activity, which capture different approaches to perform the activity. This first step requires a basic understanding of the problem to be solved. Once the appropriate agenda is selected, the further procedure is fixed to a large extent. Each step of the agenda must be performed, in an order that respects the dependencies of steps. The informal description of the step informs the software engineer about the purpose of the step. The templates associated with the step provide the software engineer with patterns that can just be filled in (which nevertheless requires creativity) or modified according to the needs of the application at hand. The result of each step is a concrete expression of the language that is used to express the artifact. If validation conditions are associated with a step, they should be checked immediately to avoid unnecessary dead ends in the development. When all steps of the agenda have been performed, a product has been developed that can be guaranteed to fulfill certain application-independent

Stage	
1	Context embedding
2	Quality requirements
3	Model construction

Figure 4: Stages of the agenda for cyclic software components

quality criteria. This product should then be subject to further validation, taking the specific application into account.

Agendas do not aim at replacing creativity, but they tell the software engineer what needs to be done and help avoid omissions and inconsistencies. Their advantage lies in an improvement of the quality of the developed products and in the possibility for reusing the knowledge incorporated in an agenda.

4 Agenda for Cyclic Software Components

The agenda for cyclic software components consists of three stages, which are shown in Figure 4. Stage 1 must be performed first; Stages 2 and 3 can be performed independently of each other. Each of the stages is performed following a sub-agenda, as described below. As a running example, we use an *intelligent cruise control* system, which serves to automatically adjust the speed of a vehicle according to the driver's request. In addition to this conventional cruise control functionality, our version uses a sensor to detect a vehicle driving ahead, and adjusts the speed to maintain a certain safety distance. This example is extracted from one of the internal ESPRESS case studies, and modified for our illustration purposes.

Stage 1: Context embedding

Embedded software is characterised by the fact that the interfaces to the environment are not standardised to a degree as it is nowadays common for software running on e.g. workstations. Hence, a developer of software for embedded systems should take special care to model the *context embedding* of the software. In ESPRESS, the context definition also serves as a starting point for a simulation of the software, using the Statemate tool. The sub-agenda for context embedding is shown in Figure 5.

Step 1.1: Specify technical interfaces.

The technical interfaces of an embedded software component are usually determined during system design, and cannot be modified by the software developer. Since their characteristics and capabilities may have significant influence on the further development, the first step of context embedding is to describe the technical interfaces in the modelling language.

Figure 5 gives templates for describing technical interfaces in our modelling language $\mu\mathcal{SZ}$. The structuring entities of $\mu\mathcal{SZ}$ are *process classes* (the outer boxes in the figure, e.g., *TechnicalDefs*), which are containers for sets of plain Z declarations, of schema definitions, and of Statemate statecharts and activity charts. The schema definitions inside a class may have assigned certain *roles*. For example, the role of

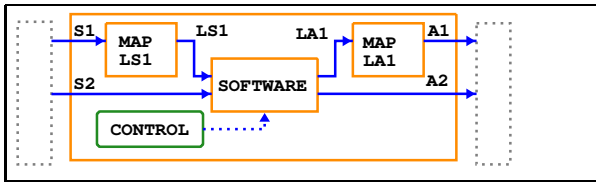
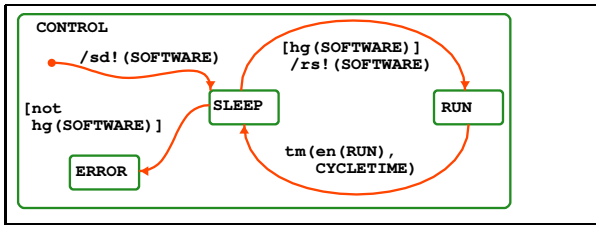
Step	Validation Conditions
1.1 Specify technical interfaces: *TechnicalDefs* [...] ... ... *TechnicalSensors* *TechnicalDefs* PORT *S*1 ... ... *TechnicalActuators* *TechnicalDefs* PORT *A*1	○ ranges of values of sensors and actuators are consistent with the technical specifications of the interfaces ○ errors of sensors and actuators are taken into account ⊢ invariants are consistent
1.2 Design and specify logical interfaces and their mapping to technical ones: *LogicalDefs* ... *LogicalSensors* *LogicalDefs* PORT *LS*1 ... *MapLS*1 *LogicalSensors*; ... PROPERTY ... *LogicalActuators* ... *MapLA*1 ...	○ errors of sensors and actuators are taken into account ⊢ invariants are consistent ⊢ mappings are unique ⊢ mappings are total
1.3 Derive software/context information flow and cycle control: *Context* *TechnicalSensors*; *TechnicalActuators* *LogicalSensors*; *LogicalActuators*   	*no conditions*

Figure 5: Steps of Stage 1: context embedding

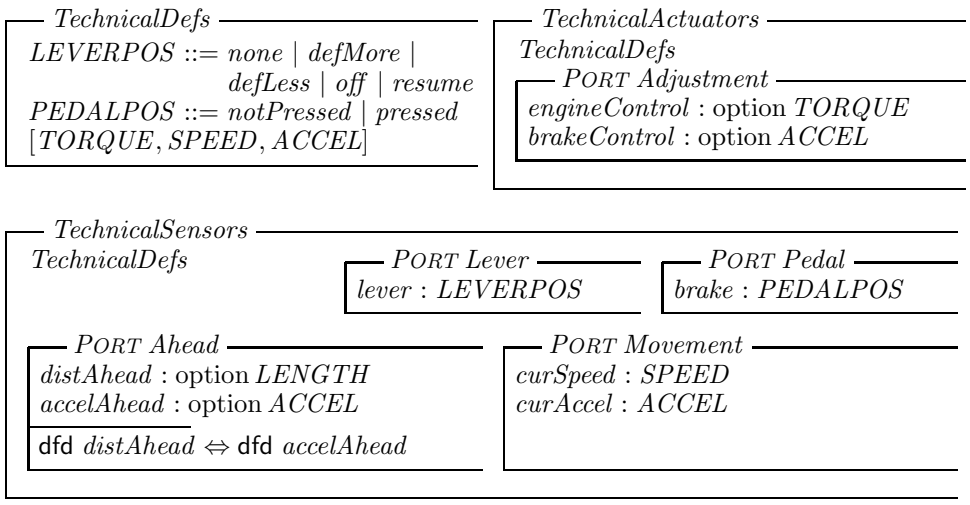


Figure 6: Technical interfaces of the cruise control

schema definitions introduced with the keyword *PORT* is to describe data variables that can be shared by a process with its environment. Interpreted standalone, *PORT* schemata do not differ from plain Z schemata. However, they contribute to the semantics of an entire process class, defining the variables belonging to the shared data state of instances of the class, as described in Section 2.

For Step **1.1**, the agenda in Figure 5 suggests to collect the technical interfaces in process classes called *TechnicalSensors* and *TechnicalActuators*, respectively, which contain sets of *PORT* schemata. The types and constants used to define these ports are collected in a third process class, *TechnicalDefs*, which is included by the other classes. The inclusion of process classes can be interpreted as textual expansion.

The validation conditions associated with Step **1.1** first require the developer to carefully check whether the types defined to model the values of sensors and actuators really capture the technical properties of the technical sensors and actuators. The second validation condition suggests to define appropriate error values for the types. Finally, all invariants must be satisfiable, i.e., there must exist legal states of the system ports, which can be checked with $\mathcal{HOL}$-$\mathcal{Z}$ in our tool environment.

Validation conditions marked with "$\circ$" are informal, whereas validation conditions marked with "$\vdash$" are formal and hence can be checked with the ESPRESS tools.

Cruise Control. Figure 6 shows how we apply Step **1.1** to the specification of the cruise control. The port *Lever* describes the driver's control lever, which can be used to turn off the cruise control, to increase or decrease the requested speed, to turn off the cruise control, and to resume its operation. The port *Pedal* models the brake pedal, the port *Ahead* the distance and relative acceleration with respect to a vehicle driving ahead, and the port *Movement* provides information about the current speed and acceleration of the vehicle. The port *Adjustment* describes the output of the cruise control, which consists of an engine torque and a (negative) acceleration for controlling the brake. Variables declared as x : option A carry values of A which may be available or not; we use $\mathsf{dfd}\ x$ to indicate whether the value of the optional variable x is available, $\mathsf{val}\ x$ to refer to that value (if it is defined), and $\mathsf{def}\ v$ to construct a defined value from v. If, e.g., the value of the sensor *distAhead* is not defined, then no vehicle driving ahead is detected, and if the actuator *engineTorque* is not defined, then the cruise control does not affect the

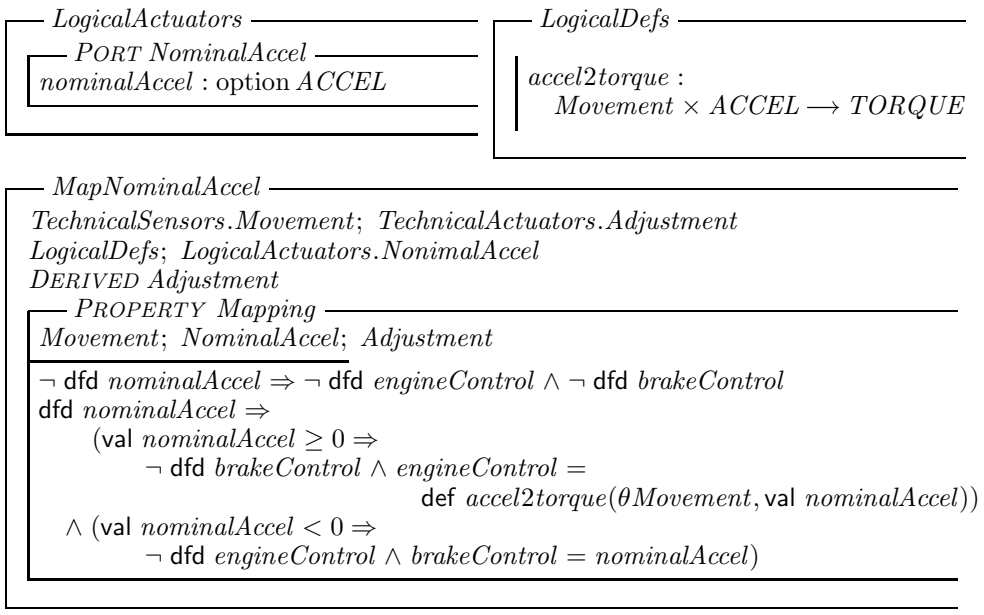


Figure 7: Logical interfaces of the cruise control

engine.

We do not present the full specification of the technical interfaces of the cruise control system here. In reality, the technical interfaces are based on the CAN bus architecture. For illustration purposes, we have introduced just one "technical" interface, which will be abstracted in the next section: the port *Adjustment*.

Step 1.2: Design and specify logical interfaces.

Apart from being non-standardised, the technical interfaces of an embedded software component may be also on a relatively low technical level, which hinders a problem-oriented specification. It may therefore be useful to introduce *abstractions* of the technical interfaces, which is achieved by defining *logical interfaces*.

The values of the logical sensors should be totally and uniquely defined by the technical sensors, and the technical actuators should be totally and uniquely defined by the logical actuators (see validation conditions associated with this step). In the simplest case, this mapping can be defined by a conversion function which maps a technical sensor to a logical sensor or a logical actuator to a technical actuator, respectively. In more complex situations, the mapping may require an internal state, for example if a logical sensor accumulates the values of a technical one. In any case, we define the mapping by a dedicated process class that describes the conversion by a property schema or by a statechart (in the template of Figure 5, these classes are called *MapLS*1 and *MapLA*1, respectively). These process classes are instantiated as sub-processes of the overall process modelling the system context, as will be seen in the next step.

The first validation condition associated with Step **1.2** suggests to apply fault tolerance techniques, e.g., consistency checks on sensor values and feedback control to check if actuator commands have been executed appropriately.

Cruise Control. We introduce a logical actuator *nominalAccel*, which abstracts from the two quite technical values *engineControl* and *brakeControl* given in the out-

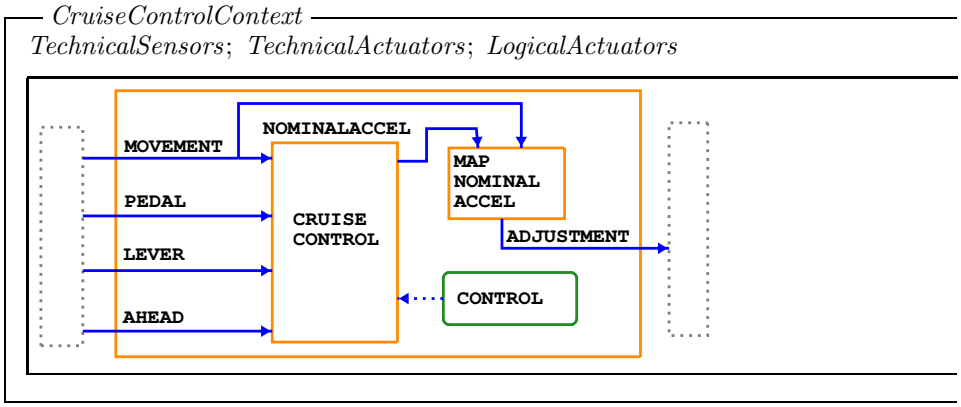


Figure 8: Activity chart of the cruise control context

put port *Adjustment*, as sketched in Figure 7. The process class *MapNominalAccel* performs the mapping of the logical to the technical actuators.

Proving the last two validation conditions amounts to proving that the function *accel2torque* defined in the class *LogicalDefs* is indeed a total function.

Step 1.3: Derive software/context information flow.

The description of the technical interfaces, the logical interfaces, and their mapping induces an activity chart, which is derived from the template given for Step **1.3** in Figure 5. The activity charts of Statemate used in $\mu\mathcal{SZ}$ combine the descriptions of information flow, of instantiation of sub-processes from process classes (rectangular boxes), and of behaviour described by statecharts (rounded boxes). In Figure 5, the overall description of the system's behaviour aggregates a sub-process *Software*, as well as sub-processes for mapping technical to logical interfaces. The information flows between these processes are labelled with ports, and semantically describe visibility of shared variables between processes. The aggregation of the statechart *Control* and the dotted lines are only for documentation purposes; they indicate that *Control* schedules the activity of the sub-process *Software*.

The scheduler defined by *Control* applies to any cyclic software component developed using this agenda, and is quite simple. It assumes that the software, once running, reads the sensors, computes the actuators, and then suspends itself. The scheduler thus periodically resumes the software in intervals of a certain cycle time. Notationally, Statemate's mechanism for suspending and resuming processes (processes are also called *activities* in Statemate) is used. The Statemate action `sd ! (SOFTWARE)` stands for suspending a process, `rs ! (SOFTWARE)` for resuming, and the condition `hg(SOFTWARE)` tests whether a process is "hanging", that is suspended. The event `tm(en(RUN),CYCLETIME)` appears *cycleTime* time units after the state `RUN` has been entered, where `en` stands for "entered", and `tm` stands for "timeout". Because the result of Step **1.3** can be derived schematically from the parts of the specification defined in Steps **1.1** and **1.2**, there are no validation conditions associated with this step.

Cruise Control. Figure 8 shows the result of Step **1.3**. We only need to draw the activity chart (where the information flows are already induced by Steps **1.1** and **1.2**); the statechart *Control* can be taken as is from the template in Figure 5. Only *Software* is renamed to *CruiseControl*.

Step	Validation Conditions
2.1 Collect relevant quality requirements.	∘ requirements are realizable
2.2 Specify model properties: *Software* *Obs*1 ... *Obs*2 ... ... *Assertion Dyn* ... ⌈ *Obs*1 ⌉ ⌢ ⌈ *Obs*2 ⌉ ...	∘ observations form a problem-oriented classification of possible situations ⊢ properties are consistent

Figure 9: Steps of Stage 2: quality requirements

Stage 2: Quality requirements

The systems we study have to fulfil certain *quality requirements*. Typical examples are safety requirements, but also high-level requirements from earlier development phases may be transferred to the software development phase. A common characteristic of quality requirements is that they only address certain *selected aspects* to be realized by the software – these aspects are important enough to be emphasised explicitly in the specification. Technically, quality requirements are formulated as *model properties*, which have to be logical consequences of the model of the software as it is constructed in Stage 3. With model properties, redundancy is deliberately introduced in the specification. This contributes to the potential for checking consistency by deduction, model checking, and systematic testing. Figure 9 describes the agenda for treating quality requirements.

Step 2.1: Collect relevant quality requirements.

The quality requirements are usually defined during system design. In this step, the ones that are relevant for the software component under development are collected and documented.

Cruise Control. A few of the quality requirements are the following:

- *Activity.* The cruise control is allowed to adjust speed only if the driver has activated it through the control lever, and did not deactivate it since then.
- *Asymptotic String Stability.* If several vehicles using the cruise control drive in a queue, a sudden change of the speed of one of them must lead to changes of speed of the the following vehicles which fade away along the queue.

Because cruise control systems are already on the market, these requirements are well understood and known to be realizable with our technical interfaces. Hence, the validation condition associated with this step is fulfilled.

Step 2.2: Specify model properties.

It is not realistic to demand that all quality requirements be specified formally as model properties. For example, the property of string stability cannot be expressed easily, because it would require to formalise aspects of the mathematics of control

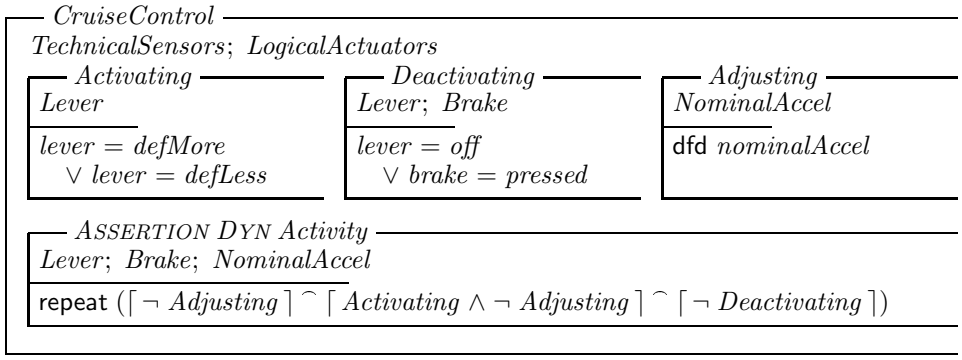


Figure 10: Model properties of the cruise control

theory[5]. However, where it is possible, the quality requirements should be expressed as model properties, to be treated automatically in a review stage later on.

A useful guideline for specifying the model properties is to first introduce abstractions of common situations observable on the interfaces. In the template of Figure 9, these are introduced by the schemata *Obs*1, *Obs*2, and applied in the temporal formula of the assertion box.

The first (informal) validation condition suggests that the model properties be oriented on a classification of the relevant situations of the observable behaviour of the system, whereas the second validation condition is an obvious consistency requirement.

Cruise Control. In Figure 10, we define schemata for observing the situations where the driver activates and deactivates the cruise control, and where the cruise control produces an output value to adjust speed. These schemata are used to formalise one of the quality requirements, namely the safety-condition "Activity". Intuitively, the temporal formula given in the property-box *Activity* can be interpreted as a kind of regular expression: the admissible traces of the behaviour of the cruise control repeatedly consists of an interval where adjustment of speed is not performed, followed by an interval where the driver activates the cruise control (adjustment of speed still does not take place), followed by an interval where the driver continuously does not deactivate the cruise control. Thereby, the temporal predicate $\lceil\, p \,\rceil$ holds for those finite or infinite intervals where the predicate p holds in each state. Note that we do not say anything about whether the cruise control actually *ever* adjusts speed; we just say when it should *not* do so. This is typical for specifying model properties, where we are only interested in selected aspects of the software.

Stage 3: Model construction

In this stage, we construct a model for the cyclic software component under consideration. There are several strategies for doing so, which depend on the problem to solve. Here we consider two variants: model construction by functional decomposition, and model construction by partitioning behaviour into operational modes.

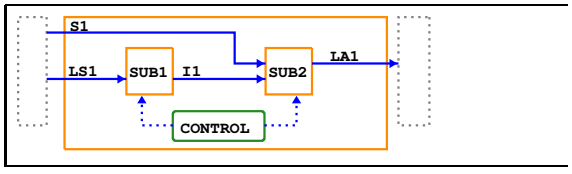
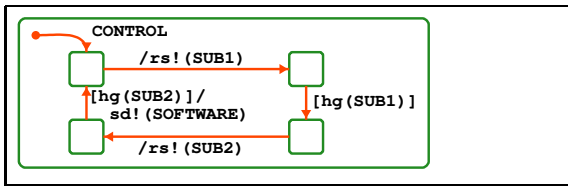
Step	Validation Conditions
3*a*.1 Design functional decomposition: *Software* *InternalInterfaces*   	⊢ information flow is free of cycles ⊢ all sensors are used and all actuators are served
3*a*.2 Specify the internal interfaces: *InternalInterfaces* PORT *I*1	⊢ invariants of ports are consistent
3*a*.3 For each subcomponent obtained in 3*a*.1, apply Stage 3 again.	*no conditions*

Figure 11: Steps of Stage 3, Variant *a*: model construction by functional decomposition

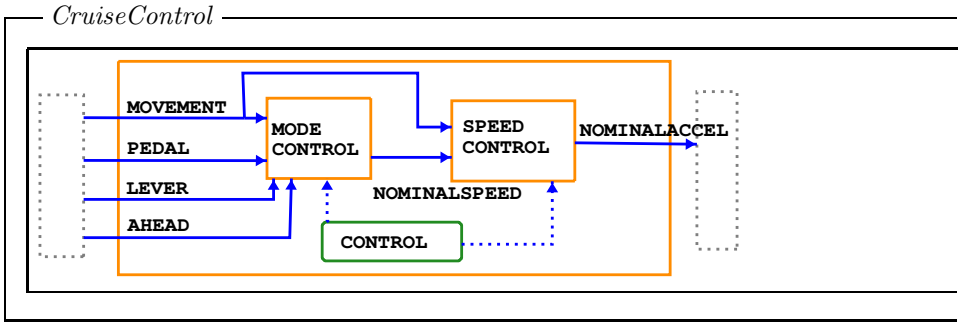


Figure 12: Functional decomposition of cruise control

Variant 3a: Model construction by functional decomposition

The problem to solve by the cyclic software component might be more adequately solved by decomposing it into subproblems, instead of giving a monolithic solution. The reasons for this may be that the problem is to large to be tackled in a monolithic way, that a decomposition follows naturally from the structure of the problem, or that existing components should be integrated into the design.

For a cyclic software component which computes output values from input values and an internal state, a decomposition is naturally achieved in a *functional* style, based on information flow between the subcomponents (Figure 11). This is also the approach the Statemate tool supports best.

Step 3a.1: Design functional decomposition using an activity chart.

Guidelines on how to perform a functional decomposition depend on the application. A useful approach is data-oriented, and considers intermediate values to be computed by the subcomponents. If we reuse existing components, these intermediate values are naturally their output interfaces. However, in general, decomposition is a problem that requires creativity. Hence our agenda suggests to first design the *principle* information flow between subcomponents by drawing an activity chart. The precise specification of the intermediate interfaces themselves is postponed until the next step[6].

Once an information flow between the subcomponents has been defined, the data dependencies canonically induce a scheduling as described by the statechart *Control* of the template for Step **3a.1** in Figure 11. Each subcomponent is treated similarly to a cyclic-software component: once it is resumed, it is expected to compute its output values and then to suspend itself. The scheduler activates the subcomponents one after the other in the order induced by the information flow.

The validation conditions associated with this step ensure that the component eventually produces an output if the subcomponents do so, and that all sensors and actuators are actually used by the system.

Cruise Control. We assume that we can reuse an existing component that implements a speed adjustment: it calculates a nominal acceleration from a given nominal speed and the vehicle movement. What is left to do is to introduce a subcomponent which controls activation and deactivation of the cruise control, and which decides

[5]In fact, notions of control theory could be expressed in Z, but Espress does not aim at these goals.

[6]This differs from the order used in Stage 1, where we first specified the interfaces, and then the information flow; but there the exact definition of the interfaces had been given by the environment.

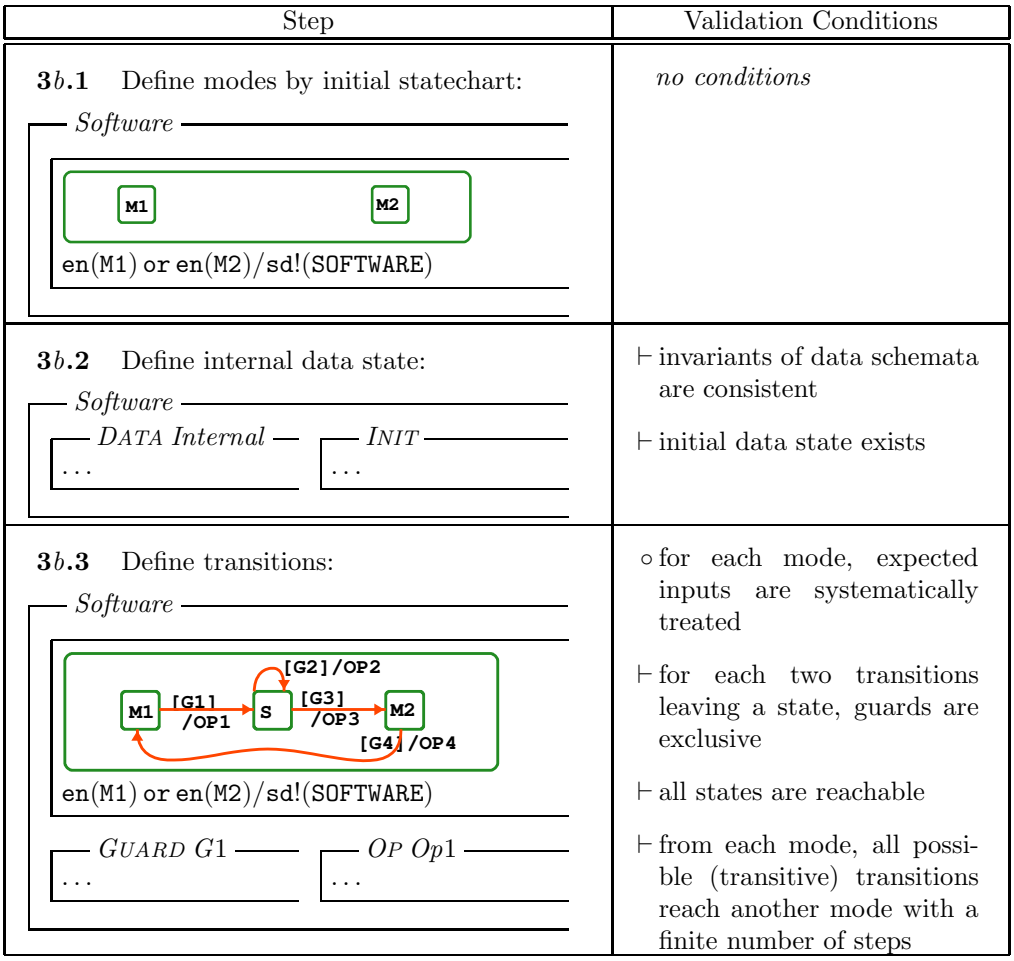


Figure 13: Steps of Stage 3, Variant b: model construction by mode-based design

to use the speed requested by the driver or a speed lower than the nominal speed to keep a certain safety distance. The decomposition leads to the activity chart given in Figure 12, where *NominalSpeed* is a newly introduced internal interface, *ModeControl* is the subcomponent controlling the activation of the cruise control, and *SpeedControl* is the reused component.

Step 3a.2: Specify the internal interfaces.

In this step, we specify the internal interfaces as they have been introduced in the last step. This step is similar to the introduction of interfaces in Stage 1; therefore, details are omitted here.

Step 3a.3: Recursively apply Stage 3.

For subcomponents yielded by the decomposition and which are not reused, we apply Stage 3 again. For the cruise control, this applies to the subcomponent *ModeControl*, which we specify using a different sub-agenda, shown in Figure 13.

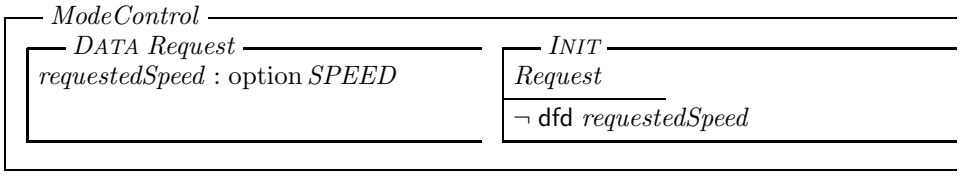


Figure 14: Internal data of the *ModeControl* subcomponent of the cruise control

Variant $3b$: Model construction by mode-based design

The problem to solve by the software component might be adequately modelled by introducing *operational modes* (such as *passive*, *active*, *emergency*, etc.). A cyclic computation then triggers transitions between the operational modes. The agenda in Figure 13 describes how to proceed for this modelling technique.

Step $3b$.1: Define modes by initial statechart.

In this step we introduce the different operational modes of the software component. Technically, this is done by defining an initial statechart (without transitions), where states or combinations of parallel states represent modes. We introduce this chart before the internal data (next step), because we might want to specify invariants on the data that depend on the current operational mode. (in $\mu\mathcal{SZ}$, the predicate instate S can be used to test whether we are in state S).

The initial statechart contains a *static reaction* (`en(M1) or en(M2)/sd!(SOFTWARE)`) which suspends the software whenever a mode is entered, signalling to the environment that the computation of the current cycle has been finished. A static reaction in Statemate is syntactically similar to a transition label `guard/action`; semantically, its action is executed whenever the guard becomes true.

Cruise Control. The complete statechart with transitions as it is obtained in Step **3b.3** is given in Figure 15, and will be explained there.

Step $3b$.2: Define internal data state.

Internal data is introduced in $\mu\mathcal{SZ}$ by a schema with the *DATA* role, its initialisation by a schema with the *INIT* role. The validation conditions associated with this step stem from the recommended Z methodology [46].

Cruise Control. In Figure 14, the internal data of the subcomponent *ModeControl* is defined. It declares a variable *requestedSpeed*, whose value (if defined) describes the nominal speed which the last time has been requested by the driver. Initially, *requestedSpeed* is undefined.

Step $3b$.3: Define transitions between states of statechart.

In this step, we refine the statechart developed in Step **3b.1** by transitions and possibly by intermediate states. Transitions are labelled systematically as `[G]/Op`, where G and Op are Z schemata introduced with the roles *GUARD* and *OP*, respectively.

Due to the static reaction introduced in Step **3b.1**, the software is suspended whenever a transition reaches a state corresponding to an operational mode. Intermediate states do not necessarily lead to a suspension, as it is the case, e.g., for the

internal state S in the template for Step **3***b*.**3** in Figure 13. The validation conditions associated with this step require the developer to check if all inputs are treated appropriately, and to show that the system behaves deterministically. Moreover, useless states that cannot be reached are not allowed. An important condition to check is whether mode transitions terminate, that is starting from any mode, for all possible inputs another mode is reached in a finite number of steps. The template state-chart given for Step **3***b*.**3** in Figure 13 shows that this condition is not trivial if intermediate states are used (it is possible that the process hangs in state S).

Cruise Control. Applying Step **3***b*.**3** to the subcomponent *ModeControl* leads to the statechart, guards and operations given in Figure 15. The statechart does not contain intermediate states. The cartesian product of the state sets $\{ACTIVATED, DEACTIVATED\}$ and $\{REQUESTED, CALCULATED\}$ makes up the set of operational modes. In addition to the declared objects, we use the following Z constants: $stepSpeed : SPEED$ is the offset how to increase or decrease the requested speed, and $maxSpeed : SPEED$ is the maximum requested speed the cruise control is allowed to manage. The function $safeDistance : SPEED \longrightarrow LENGTH$ yields the safe distance to a vehicle ahead in dependency of a driving speed. The function $distanceRegulator : Movement \times Ahead \longrightarrow SPEED$ represents an algorithm calculating a nominal speed from the movement of the vehicle and information about a vehicle ahead.

Discussion. We have demonstrated that the agenda approach supports the systematic development of requirement specifications for high quality embedded systems on a non-trivial level of detail which gives substantial guidance to developers. As already noted, agendas are not intended to replace creativity and do not aim at completely automating development processes. Hence, in the first steps of an agenda, high-level decisions have to be taken. The validation conditions associated with the early steps of an agenda are mostly informal, encouraging developers to carefully re-consider their decisions, see e.g. Step 1.1 of the agenda of Figure 5. Later steps in an agenda, on the other hand, often have validation conditions associated with them that can be formally expressed and proven. The reason is that in the later steps consistency conditions between the various parts of the specification that are already developed can be stated. Step 1.2 of the agenda shown in Figure 5 is an example. Finally, some steps of an agenda (usually the last steps) can be performed in an entirely schematic way, because they merely consist in an appropriate combination of parts of the specification developed in earlier steps, see e.g. Step 1.3 of the agenda of Figure 5.

5 Related Work

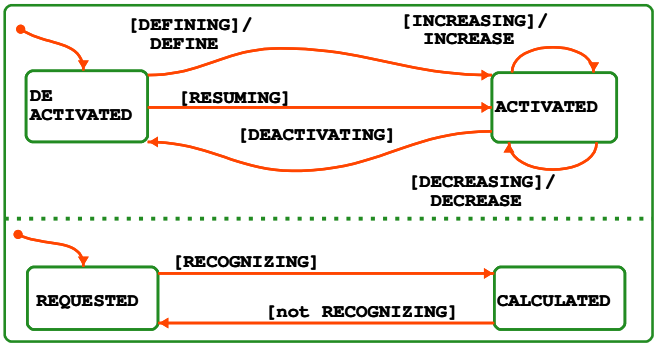
Our work relates to methodological aspects of software engineering in general, and to approaches to formally specify safety-critical systems.

5.1 Development Methods

Other approaches to make software engineering more systematic can be divided into product-oriented and process-oriented approaches.

Product-oriented approaches. Recently, efforts have been made to support re-use of special kinds of software development knowledge: *Design patterns* [12] have had much success in object-oriented software construction. They represent frequently used ways to combine classes or associate objects to achieve a certain

ModeControl



`en(DEACTIVATED) or`
`en(ACTIVATED) or`
`en(CALCULATED) or`
`en(REQUESTED)`
`/ sd!(MODECONTROL)`

GUARD Defining

Lever; *Pedal*

$lever \in \{defMore, defLess\}$
$brake = notPressed$

OP Define

Movement; $\Delta Request$

$requestedSpeed' = \mathsf{def}\ currSpeed$

GUARD Increasing

Lever; *Pedal*

$lever = defMore \wedge brake = notPressed$

OP Increase

$\Delta Request$

$requestedSpeed' =$
$\mathsf{def}\ \min\{\mathsf{val}\ requestedSpeed + stepSpeed,$
$maxSpeed\}$

GUARD Decreasing

Lever; *Pedal*

$lever = defLess \wedge brake = notPressed$

OP Decrease

$\Delta Request$

...

GUARD Resuming

Lever; *Pedal*

$\mathsf{dfd}\ requestedSpeed$
$lever = resume$
$brake = notPressed$

GUARD Deactivating

Lever; *Pedal*

$lever = off$
$\vee\ brake = pressed$

GUARD Recognizing

Ahead; *Movement*

$\mathsf{dfd}\ distAhead$
$\mathsf{val}\ distAhead <$
$safeDistance(currSpeed)$

DERIVED NominalSpeed

PROPERTY

Movement; *Ahead*; *Request*; *NominalSpeed*; CONTROL

$\mathsf{instate}\ DEACTIVATED \Rightarrow \neg\ \mathsf{dfd}\ nominalSpeed$
$\mathsf{instate}\ ACTIVATED \wedge \mathsf{instate}\ REQUESTED \Rightarrow nominalSpeed = requestedSpeed$
$\mathsf{instate}\ ACTIVATED \wedge \mathsf{instate}\ CALCULATED \Rightarrow$
$nominalSpeed = \mathsf{def}\ \min\{distanceRegulator(\theta Movement, \theta Ahead),$
$\mathsf{val}\ requestedSpeed\}$

Figure 15: Modes and transitions of the *ModeControl* subcomponent of the cruise control

purpose. Furthermore, in the field of software architecture [39], *architectural styles* have been defined that capture frequently used design principles for software systems. Apart from the fact that these concepts are more specialised in their application than agendas, the main difference is that design patterns and architectural styles do not describe *processes* but *products*.

A prominent example of knowledge-based software engineering, whose aims closely resemble our own, is the Programmer's Apprentice project [38]. There, programming knowledge is represented by *clichés*, which are prototypical examples of the artifacts in question. The programming task is performed by "inspection"– i.e., by choosing an appropriate cliché and customising it. In comparison to clichés, agendas are more process-oriented.

Process-oriented approaches. Chernack [7] uses a concept called *checklist* to support inspection processes. In contrast to agendas, checklists presuppose the existence of a software artifact and aim at detecting defects in this artifact.

Potts [35] aims at capturing not only strategic but also heuristic aspects of design methods. He uses *Issue-Based Information Systems* (IBIS) [8] as a representation formalism for design methods. IBIS representing heuristics tend to be specialised for particular application domains, and it is not clear how much of an IBIS can be re-used for similar applications.

Johnson and Feather [26] take a transformational approach to supporting the specification process. Starting out from a simple initial specification, *evolution transformations* are applied. These may change the semantics of the specification and add more detail to it. Compared to these, agendas are concepts of a higher level of abstraction and closer to human reasoning.

Related to our aim to provide methodological support for applying formal techniques is the work of Souquières and Lévy [40]. They support specification acquisition with *development operators* that reduce *tasks* to subtasks. However, their approach is limited to specification acquisition, and the development operators do not provide means to validate the developed specification.

Astesiano and Reggio [1] also emphasise the importance of method when using formal techniques. They set up a *method pattern* for formal specification, consisting of context, formalism, and pragmatics. Pragmatics, in turn, contain rationale, guidelines, presentation, and documentation. Guidelines (see [37] for an example) correspond to the steps of an agenda (i.e., an agenda without dependency graph, templates, and validation conditions).

Wile's [43] development language Paddle provides a means of describing procedures for transforming specifications into programs. Since carrying out a process specified in Paddle involves executing the corresponding program, one disadvantage of this procedural representation of process knowledge is that it enforces a strict depth-first left-to-right processing of the goal structure. The "second generation" process language JIL [42] is more flexible in this respect: not only proactive (i.e., fixed) but also reactive control specifications can be given. Representing processes as programs has the advantage that processes are formal objects on which reasoning can be performed. On the other hand, humans find it easier to follow instructions when they are expressed on a higher level of abstraction than a programming notation can provide.

Process languages are widely used in the field of software process modelling [10, 11]. Software process modelling aims at automated support for software development processes on the level of company processes. The goal is to execute software processes like software [33]. This goal implies that the language used to express processes should be similar to a programming language. Typical activities that are considered in software process modelling are filling in forms, sending emails,

using a version control system to check in or check out files, or starting an editor or another tool. In contrast, agendas do not consider company processes but single documents and their semantics. They structure activities that need creativity and do not lend themselves easily to automatic enactment. Therefore, programming language notations are not necessary to express them. All in all, agendas describe activities that are considered atomic in software process modelling. Hence, process modelling techniques are a useful complement to agendas. To obtain realistic processes for larger projects, they should always be used in combination.

Humphrey [24] has devised the *personal software process* to improve the performance of software engineers. Software engineers are trained to plan and analyze their activities and to perform measurements on them. Personal strengths and weaknesses are identified. Based on this knowledge, software engineers improve their personal skills. The personal software process approach is centred around individuals, not around activities, as agendas are. As with process modelling techniques, the two approaches do not exclude but complement each other. The methods captured in agendas can only be worked out by highly competent individuals.

5.2 Safety-critical systems

Moser's and Melliar-Smith's approach to the formal verification of safety-critical systems [31] comprises the specification, design and implementation phases. They use a reliability model for the processors that execute the program. This enables them to take computer failures into account, an aspect we do not address. On the other hand, their approach does not cover the validation of the top-level specification, an issue that we pay particular attention to.

The use of model-based languages like Z or VDM [27] in the area of system safety has been thoroughly investigated. Several case studies use VDM, e.g. the British government regulations for storing explosives [32], a railway interlocking system [15], and a water-level monitoring system [44]. Mukherjee's and Stavridou's as well as Hansen's work, however, focus on adequately modelling safety requirements, independently of the question of whether software is employed or not. Consequently, they do not discuss issues specific to the construction of safety-critical software.

Jacky [25] uses Z to define a framework for safety-critical systems that emphasises safety interlocking. McDermid and Pierce [29] define a graphical notation based on a variant of statecharts [17] that is translated into Z for the purpose of mechanical validation. This notation is used to specify and develop software for programmable logic controllers. Halang and Krämer [14] also focus on programmable logic controllers. They describe a development process, from the formalisation of requirements to the testing of the constructed program. They use the specification language Obj and the Hoare calculus, and their choice is motivated by the available tool support. The specification language Obj is weaker than $\mu\mathcal{SZ}$ because Obj only allows conditional equations to be stated.

Ravn et al. [36] use the duration calculus to express functional requirements and safety constraints. The Duration Calculus is a real-time interval logic based on state durations. A system is modelled by a collection of state variables that are functions of time, modelled by real numbers. System requirements are specified by means of duration formulas that are predicates containing integrals over state assertions (predicates on state variables). This form of specification allows to specify requirements like critical durations of system states or progress and stability requirements. Its most serious drawbacks are, first, that the system state is constituted by a plain set of state variables. Second, (at least in the requirements specification) no means are provided to hierarchally decompose a system.

Heisel and Sühl [22, 20] use a combination of Z and real-time CSP [9] to formally specify software for embedded real-time systems. Their reference architectures and

agendas are similar to the one used in ESPRESS.

6 Conclusions

Agendas provide more than a process description. They are an overall documentation of processes and products, and they are easily adjusted to similar problems. Agendas together with templates combine both process and product oriented approaches (see Section 5.1).

Besides providing guidance for developers and ensuring some application independent quality aspects of the developed product, agendas offer the following advantages:

- *Agendas make software processes explicit, comprehensible, and assessable.* Giving concrete steps to perform an activity and defining the dependencies between the steps make processes explicit. The process becomes comprehensible for third parties because the purpose of the various steps is described informally in the agenda. Thus, agendas may be subject to evaluation.

- *Agendas standardise processes and products of software development.* Agendas structure development processes. The development of an artifact following an agenda always proceeds in a way consistent with the steps of the agenda and their dependencies. Thus, processes supported by agendas are standardised. The same holds for the products: since applying an agenda results in instantiating the templates given in the agenda, all products developed with an agenda have a similar structure.

- *Agendas support maintenance and evolution of the developed artifacts.* Understanding a document developed by another person is less difficult when the document was developed following an agenda than without such information. Each part of the document can be traced back to a step in the agenda, which reveals its purpose. To change the document, the agenda can be "replayed". The agenda helps focus attention on the parts that actually are subject to change.

- *Agendas are a promising starting point for sophisticated machine support.* They can form the basis of a process-centred software engineering environment (PSEE) [13]. Such a tool would lead its users through the process described by the agenda.

Agendas are language-independent to a large extent. Changing the language in which the developed specification is expressed consists mostly in replacing the templates of the various steps, and effects the steps themselves very little, see the agenda presented in [21].

The validation conditions are a very important aspect of agendas. Clearly, the errors revealed by failing to demonstrate validations conditions of an agenda can only be of an application-independent nature. Checking the validation conditions cannot guarantee, e.g., that a system is adequately modelled by a developed specification. Nevertheless, many common errors can be discovered. As reported by Heitmeyer et al. [23], in the certification of the Darlington plant (which cost $ 40M), "the reviewers spent too much of their time and energy checking for simple, application-independent properties." To improve this situation, Heitmeyer et al. have implemented a tool that performs consistency checks. Since this tool is not tailored for any application domain, it can only check very general consistency conditions. In comparison, the validation conditions provided by agendas are much more to the point (see e.g. the validation conditions of Step 3.b.3 of the agenda

shown in Fig. 13), such that more specific tool support for checking validation conditions generated by agendas is conceivable. But even if no specific support tools for agendas are available, agendas allow developers to use existing tools, e.g., Statemate to check the specification by simulation, or type checkers and theorem provers for Z to check some of the formal validation conditions.

For these reasons, agendas play a central role in the ESPRESS methodology, and we have used the agenda approach for supporting further activities of the general development process as shown in Fig. 1. An agenda has been developed for the activity of safety analyses, which is based on common techniques such as FTA (failure-tree analysis) and SHARD (software hazard analysis and resolution design).

Cyclic software components, though important in practice, are indeed a rather simple software architecture. An extension of our agenda for this architecture to certain kinds of event-triggered software components, which are also studied in the case studies of ESPRESS [3], has been developed in [45]. Significant parts of the agenda given in this paper, in particular from Stage 1, context embedding, and Stage 2, quality requirements, have been reused.

Acknowledgments.

Many results and ideas presented in this paper stem from the broader context of the ESPRESS project and the work of its many collaborators in Berlin. We would especially like to thank Mirco Conrad and Eckard Lehmann for their work on the cruise control case study, and Thomas Santen for his comments on a draft of this paper.

References

[1] E. Astesiano and G. Reggio. Formalism and Method. In M. Bidoit and M. Dauchet, editors, *Proc. TAPSOFT'97*, LNCS 1214, pages 93–114. Springer-Verlag, 1997.

[2] R. Büssow, H. Dörr, R. Geisler, W. Grieskamp, and M. Klar. μSZ – ein Ansatz zur systematischen Verbindung von Z und Statecharts. Technical Report TR 96-32, Technische Universität Berlin, 1996.

[3] R. Büssow, R. Geisler, and M. Klar. Specifying safety-critical embedded systems with statecharts and Z: a case study. In E. Astesiano, editor, *Proc. ETAPS-FASE'98*, LNCS 1382, pages 71–87. Springer-Verlag, 1998.

[4] Robert Büssow, Robert Geisler, Wolfgang Grieskamp, and Marcus Klar. The $\mu\mathcal{SZ}$ notation version 1.0. Technical Report 97–26, Technische Universität Berlin, Fachbereich Informatik, December 1997.

[5] Robert Büssow and Wolfgang Grieskamp. Combinig Z and temporal interval logics for the formalization of properties and behaviors of embedded systems. In R. K. Shyamasundar and K. Ueda, editors, *Advances in Computing Science – Asian '97*, LNCS 1345. Springer-Verlag, 1997.

[6] Robert Büssow, Wolfgang Grieskamp, Winfried Heicking, and Stephan Hermann. An open environment for the integration of heterogeneous modelling techniques and tools. In *Current Trends in Applied Formal Methods*, LNCS, Boppard, Germany, October 1998.

[7] Y. Chernack. A statistical approach to the inspection checklist formal synthesis and improvement. *IEEE TSE*, 22(12):866–874, December 1996.

[8] J. Conclin and M. Begeman. gIBIS: a hypertext tool for exploratory policy discussion. *ACM Transactions on Office Informations Systems*, 6:303–331, October 1988.

[9] Jim Davies. *Specification and Proof in Real-Time CSP*. Cambridge University Press, 1993.

[10] A. Finkelstein, J. Kramer, and B. Nuseibeh, editors. *Software Process Modelling and Technology*. Wiley, 1994.

[11] A. Fuggetta and A. Wolf, editors. *Software Process*. Trends in Software 4. Wiley, 1996.

[12] E. Gamma, R. Helm, R. Johnson, and Vlissides. J. *Design Patterns – Elements of Reusable Object-Oriented Software*. Addison Wesley, Reading, 1995.

[13] P. Garg and M. Jazayeri. Process-centered software engineering environments: A grand tour. In A. Fuggetta and A. Wolf, editors, *Software Process*, number 4 in Trends in Software, chapter 2, pages 25–52. Wiley, 1996.

[14] Wolfgang Halang and Bernd Krämer. Safety assurance in process control. *IEEE Software*, 11(1):61–67, January 1994.

[15] Kirsten Mark Hansen. Modelling railway interlocking systems. Available via ftp from ftp.ifad.dk, directory /pub/vdm/examples, 1994.

[16] D. Harel, H. Lachover, A. Naamad, A. Pnueli, M. Politi, R. Sherman, A. Shtull-Trauring, and M. Trakhtenbrot. Statemate: A working environment for the development of complex reactive systems. *IEEE Transactions on Software Engineering*, 16 No. 4, April 1990.

[17] David Harel. Statecharts: a visual formalism for complex systems. *Science of Computer Programming*, 8:231–274, 1987.

[18] David Harel and Amnon Naamad. The STATEMATE semantics of statecharts. *ACM Trans. Soft. Eng. Method.*, October 1996.

[19] M. Heisel. *Methodology and Machine Support for the Application of Formal Techniques in Software Engineering*. Habilitation Thesis, TU Berlin, 1997.

[20] M. Heisel and C. Sühl. Methodological support for formally specifying safety-critical software. In P. Daniel, editor, *Proceedings 16th International Conference on Computer Safety, Reliability and Security (SAFECOMP)*, pages 295–308. Springer-Verlag London, 1997.

[21] Maritta Heisel. Agendas – a concept to guide software development activites. In R. N. Horspool, editor, *Proc. Systems Implementation 2000*, pages 19–32, London, 1998. Chapman & Hall.

[22] Maritta Heisel and Carsten Sühl. Formal specification of safety-critical software with Z and real-time CSP. In E. Schoitsch, editor, *Proceedings 15th International Conference on Computer Safety, Reliability and Security (SAFECOMP)*, pages 31–45. Springer-Verlag London, 1996.

[23] C. Heitmeyer, R. Jeffords, and B. Lebaw. Automated consistency checking of requirements specifications. *ACM Transactions on Software Engineering and Methodology*, 5(3):231–261, July 1996.

[24] Watts S. Humphrey. *A Discipline for Software Engineering*. Addison-Wesley, 1995.

[25] Jonathan Jacky. Specifying a safety-critical control system in Z. *IEEE Transactions on Software Engineering*, 21(2):99–106, February 1995.

[26] W. Lewis Johnson and Martin S. Feather. Using evolution transformations to construct specifications. In Michael R. Lowry and Robert D. McCartney, editors, *Automating Software Design*, pages 65 – 92. AAAI Press, 1991.

[27] Cliff B. Jones. *Systematic Software Development using VDM*. Prentice Hall, 1990.

[28] Kolyang, T. Santen, and B. Wolff. A structure preserving encoding of Z in Isabelle/HOL. In J. von Wright, J. Grundy, and J. Harrison, editors, *Theorem Proving in Higher-Order Logics*, LNCS 1125. Springer-Verlag, 1996.

[29] J.A. McDermid and R.H. Pierce. Accessible formal method support for PLC software development. In G. Rabe, editor, *Proceedings of the 14th International Conference on Computer Safety, Reliablity and Security (SAFECOMP), Belgirate, Italy*, pages 113–127, London, 1995. Springer-Verlag.

[30] Kenneth L. McMillan. *Symbolic Model Checking*. Kluwer Academic Publishers, 1993.

[31] Louise E. Moser and P.M. Melliar-Smith. Formal verification of safety-critical systems. *Software – Practice and Experience*, 20(8):799–821, August 1990.

[32] Paul Mukherjee and Victoria Stavridou. The formal specification of safety requirements for storing explosives. *Formal Aspects of Computing*, 5:299–336, 1993.

[33] Leon Osterweil. Software processes are software too. In *9th International Conference on Software Engineering*, pages 2–13. IEEE Computer Society Press, 1987.

[34] L. C. Paulson. *Isabelle*. LNCS 828. Springer-Verlag, 1994.

[35] Colin Potts. A generic model for representing design methods. In *International Conference on Software Engineering*, pages 217–226. IEEE Computer Society Press, 1989.

[36] A.P. Ravn, H. Rischel, and K.M. Hansen. Specifying and verifying requirements of real-time systems. *IEEE Transactions on Software Engineering*, 19(1):41–55, January 1993.

[37] Gianna Reggio. A method to capture formal requirements: the INVOICE case study. In M. Allemand, C. Attiogbe, and H. Habrias, editors, *Proceedings Workshop on Comparing Specification Techniques*. IRIN – Université de Nantes, 1998.

[38] Charles Rich and Richard C. Waters. The programmer's apprentice: A research overview. *IEEE Computer*, pages 10–25, November 1988.

[39] M. Shaw and D. Garlan. *Software Architecture*. IEEE Computer Society Press, Los Alamitos, 1996.

[40] Jeanine Souquières and Nicole Lévy. Description of specification developments. In *Proc. of Requirements Engineering '93*, pages 216–223. IEEE Computer Society Press, 1993.

[41] J.M. Spivey. *The Z Notation – A Reference Manual*. Prentice Hall, 1992.

[42] Stanley Sutton and Leon Osterweil. The design of a next-generation process language. In M. Jazayeri and H. Schauer, editors, *Proceedings of the Fifth ACM SIGSOFT Symposium on the Foundations of Software Engineering*, LNCS 1301, pages 142–158. Springer-Verlag, 1997.

[43] David S. Wile. Program developments: Formal explanations of implementations. *Communications of the ACM*, 26(11):902–911, November 1983.

[44] Lloyd Williams. Assessment of safety-critical specifications. *IEEE Software*, pages 51–60, January 1994.

[45] Kirsten Winter, Thomas Santen, and Maritta Heisel. An agenda for event-driven software components with complex data models. In W. Ehrenberger, editor, *Proceedings of the 16th International Conference on Computer Safety, Reliablity and Security (SAFECOMP)*, LNCS 1516, pages 16–31. Springer-Verlag, 1998.

[46] J. B. Wordsworth. *Software Development with Z.* Addison-Wesley, 1992.